KIGALI COLLEGE

- Option: INFORMATION TECHNOLOGY
- Module: BACKEND DEVELOPMENT USING JAVA
- Reg Number: 24RP05481
- Names: Divine AKISA
- Class: Y2 IT
. Dates: 17 Feb, 2026

FINANCIAL TRANSACTION TRACKING SYSTEM REPORT

1. PROJECT OVERVIEW

a) Project Proposal

Project Title: Financial Transaction Tracking System (FTTS)

Problem Statement: Manual tracking of financial transactions is error-prone and inefficient. Existing solutions often lack proper security, audit trail, and real-time monitoring, making it difficult to prevent fraud and ensure data integrity.

Objectives:

General Objective: To develop a secure and efficient system to track financial transactions.

Specific Objectives:

- Automate transaction recording and approval workflow.
- Implement role-based access control.
- Ensure data integrity and audit logging.
- Detect suspicious or fraudulent activities.
- Provide user-friendly dashboards and reports.

Scope of the System

- Handles user registration and authentication.
- Manages accounts, categories, and transactions.
- Tracks and logs all user activities.
- Supports transaction approval workflows.
- Provides alerts and notifications.

Technologies Used:

- Java (JSP, Servlets)

- MySQL
- Apache Tomcat
- Bootstrap for UI
- BCrypt for password hashing
- Chart.js for dashboard graphs

Expected Outcomes:

- Efficient transaction tracking.
- Secure access and role-based controls.
- Real-time balance updates.
- Fraud detection and alerts.
- Comprehensive reports.

b) System Analysis Document

Functional Requirements:

- User registration and login.
- Account creation and management.
- Transaction creation, approval, and tracking.
- Notifications and alerts for suspicious activities.
- Dashboard displaying summaries.

Non-functional Requirements:

- Security (SQL Injection, XSS, CSRF prevention)
- Performance (Quick transaction processing)
- Usability (Intuitive UI, responsive design)

- Reliability (Data integrity, ACID compliance)

User Roles:

- Admin: Full access to system configuration.
- Manager: Approve high-value transactions.
- User: Create transactions and view own accounts.

Use Case Descriptions:

1. **User Registration:** User submits personal details => System validates => Account created.

2. **Transaction Creation:** User enters transaction => System validates balance => Pending or completed.

3. **Transaction Approval:** Manager reviews pending transactions => Approves or rejects => System updates balances.

4. **Notifications:** System sends alerts for suspicious activity.

c) System Design

System Architecture Diagram (MVC):

- Model: JavaBeans representing Users, Accounts, Transactions, Categories, AuditLogs, Notifications.
- View: JSP pages for UI.
- Controller: Servlets handling requests and responses.

Use Case Diagram:

- Actors: User, Manager, Admin
- Actions: Register, Login, Create Transaction, Approve Transaction, View Reports

Class Diagram:

- Classes: User, Account, Transaction, Category, AuditLog, Notification
- Relationships: User => Account => Transaction

Sequence Diagram:

- User submits transaction => TransactionServlet validates => TransactionDAO updates database => AuditLogDAO records action=> Response sent to user

Database ER Diagram:

- Tables: users, accounts, categories, transactions, transfers, audit_log, suspicious_activities, notifications, sessions
- Relationships: Foreign keys, indexes, constraints

2. Database Requirements

Database Name: ftts_db

Tables:

1. users (PK: user_id)
2. accounts (PK: account_id, FK: user_id)
3. categories (PK: category_id)
4. transactions (PK: transaction_id, FK: account_id, category_id)
5. transfers (PK: transfer_id, FK: transaction_id, account_id)
6. audit_log (PK: log_id, FK: user_id, transaction_id)
7. suspicious_activities (PK: activity_id, FK: user_id, transaction_id)
8. notifications (PK: notification_id, FK: user_id)
9. sessions (PK: session_id, FK: user_id)

SQL Scripts:

- CREATE DATABASE ftts_db;

- CREATE TABLE users (...);
- INSERT INTO categories (...); - Normalized to 3NF.

3. Application Implementation (Core)

a) Source Code

- **JavaBeans (Model):** User.java, Account.java, Transaction.java, Category.java, AuditLog.java, Notification.java
- **DAO Classes:** UserDAO.java, AccountDAO.java, TransactionDAO.java, CategoryDAO.java, AuditLogDAO.java
- **Servlets (Controller):** LoginServlet.java, RegisterServlet.java, LogoutServlet.java, TransactionServlet.java, AccountServlet.java
- **JSP Pages (View):** index.jsp, login.jsp, register.jsp, dashboard.jsp, accounts.jsp, error pages
- **JDBC connection:** DatabaseUtil.java
- Proper MVC package structure

b) CRUD Operations

- Create, Read, Update, Delete implemented for Users, Accounts, Transactions, Categories.

c) Authentication & Authorization

- Login page with session management
- Role-based access control

4. Business Logic Implementation

- Validations: Server-side validation for email, phone, amount, PIN.
- Business rules: High-value transactions require manager approval.
- Status changes: pending =>approved/rejected
- Error handling & messages: Displayed on JSP pages

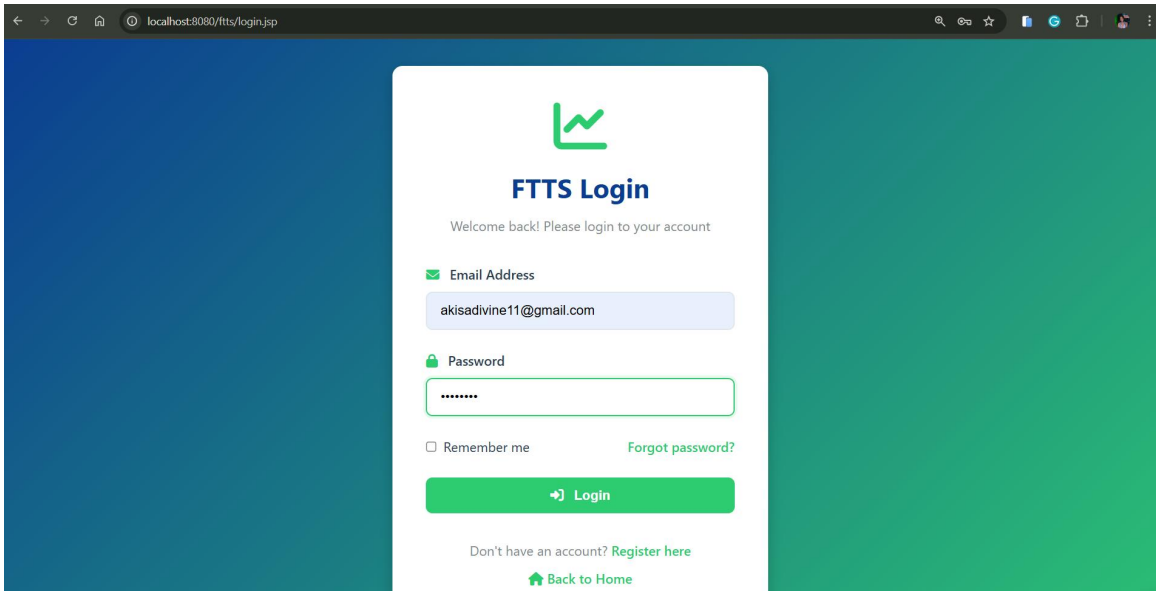
5. User Interface (UI)

- Forms, tables, buttons for Add/Edit/Delete
- Clean layout with Bootstrap
- User-friendly navigation and dashboard

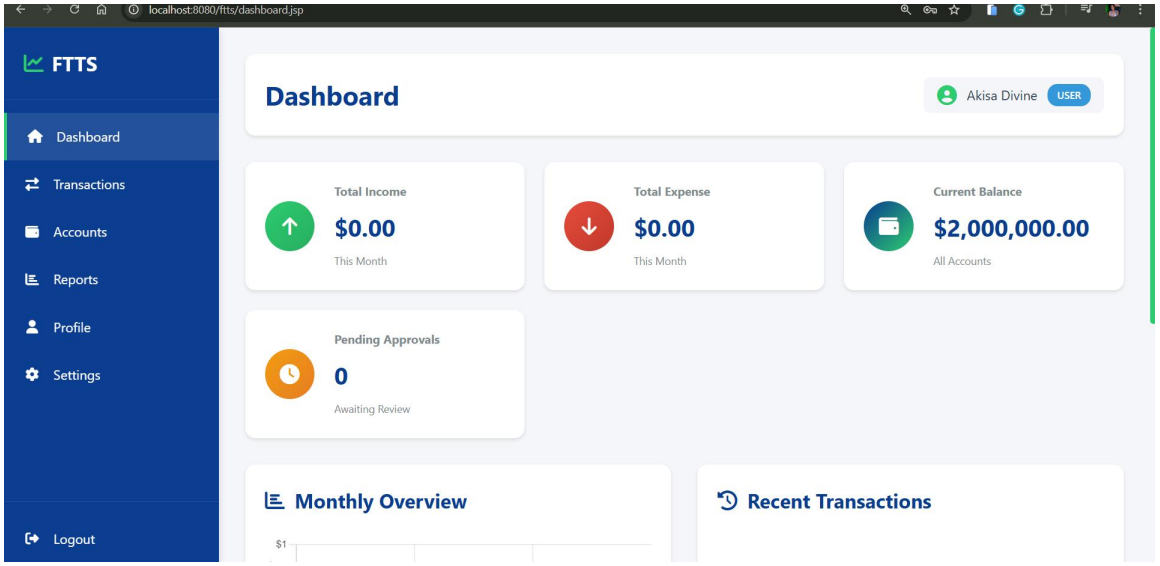
6. Testing Evidence

- Test cases for all modules
    - Login success/failure

First I entered The Correct Data



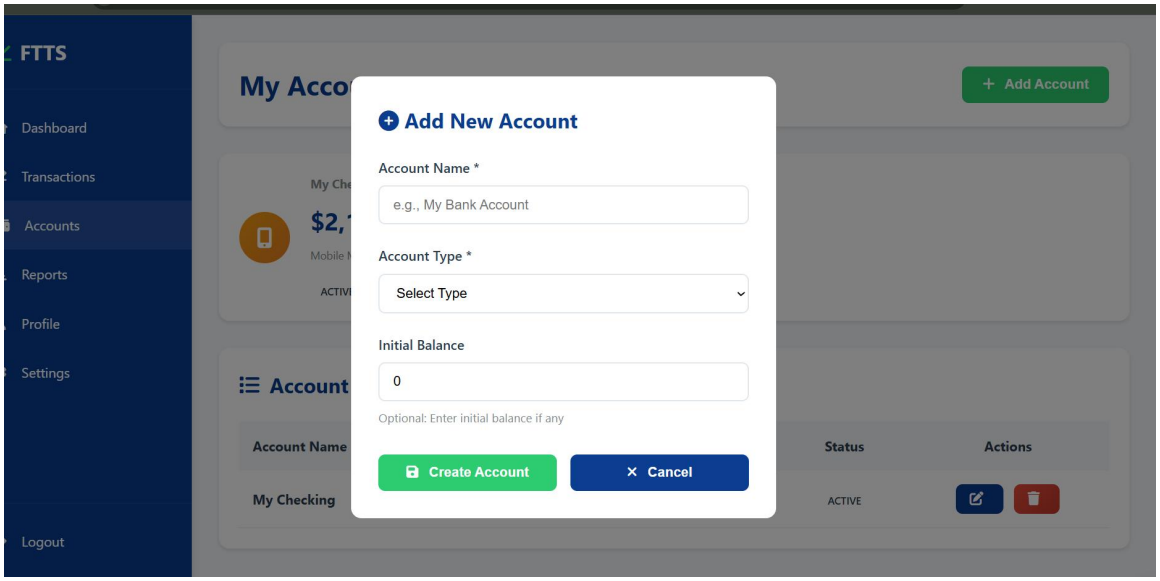
Click On the Login throw me on the dashboard

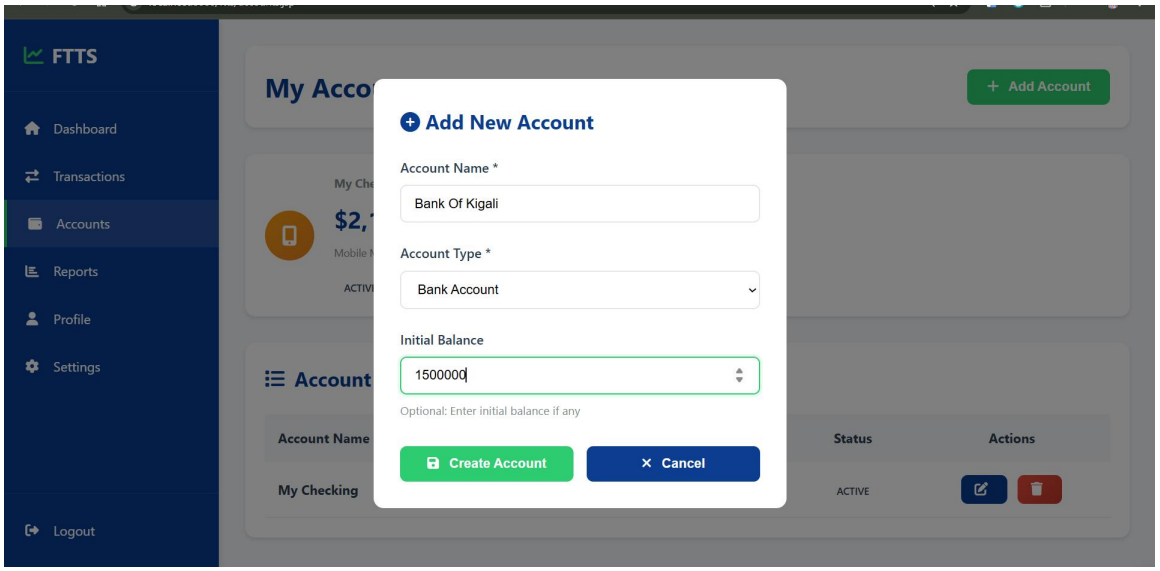


- CRUD operations

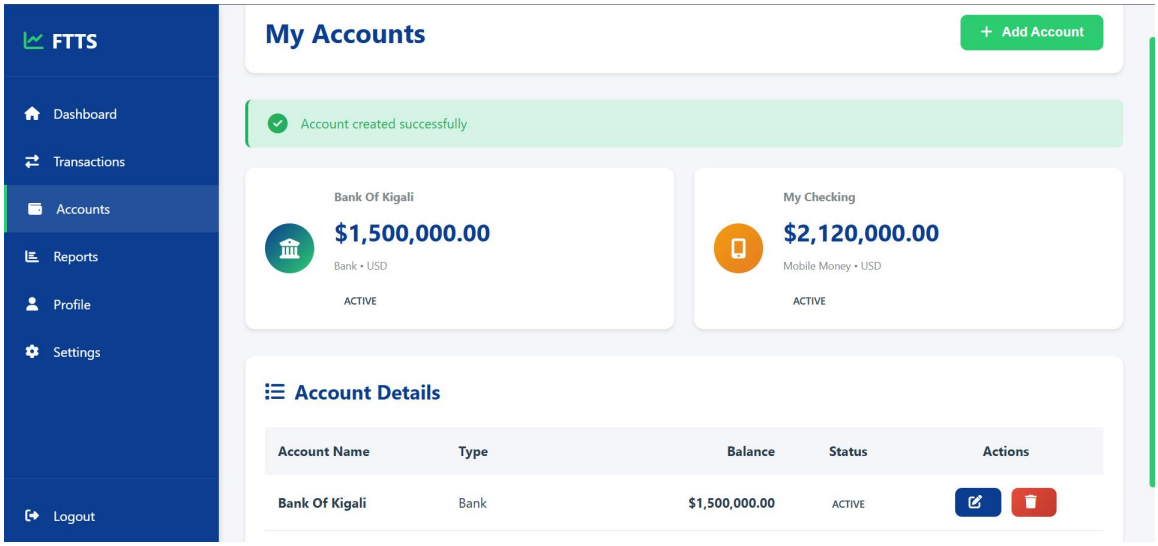
Create Operation

Add New Account After Login



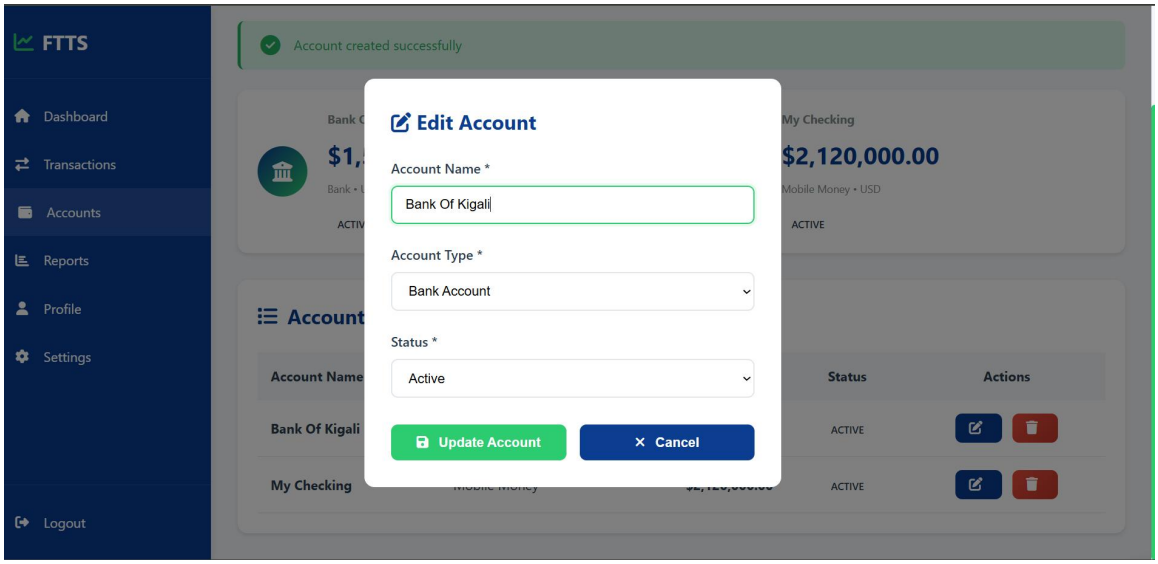


Now Account Created Successful

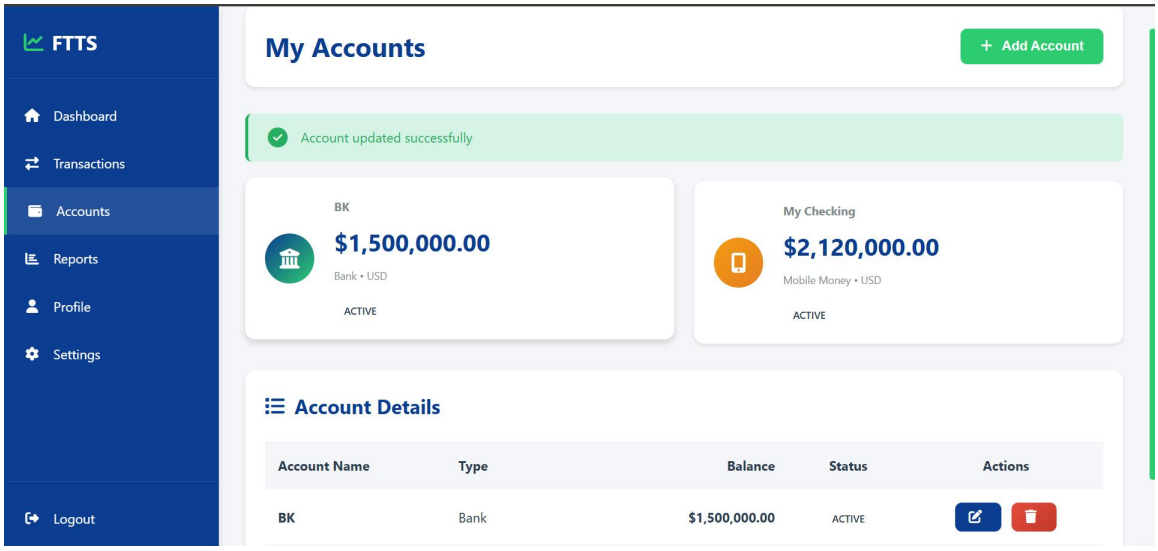


Update Account

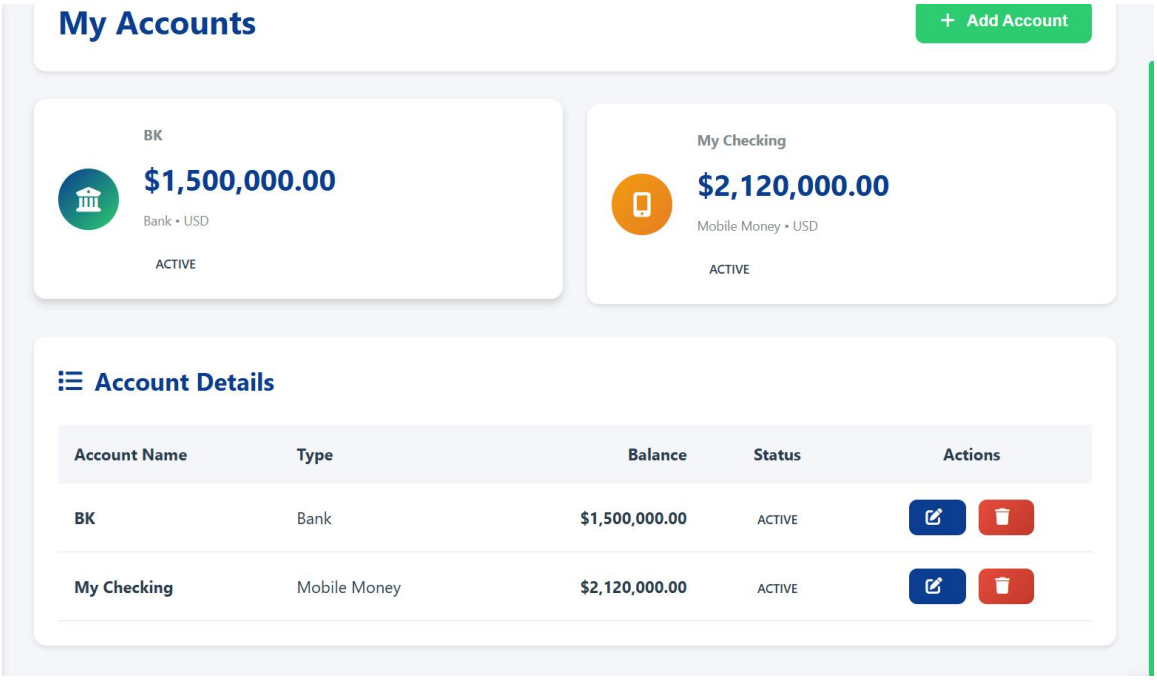
Before Update Account Name is Bank Of Kigali let's change it to BK



After Update

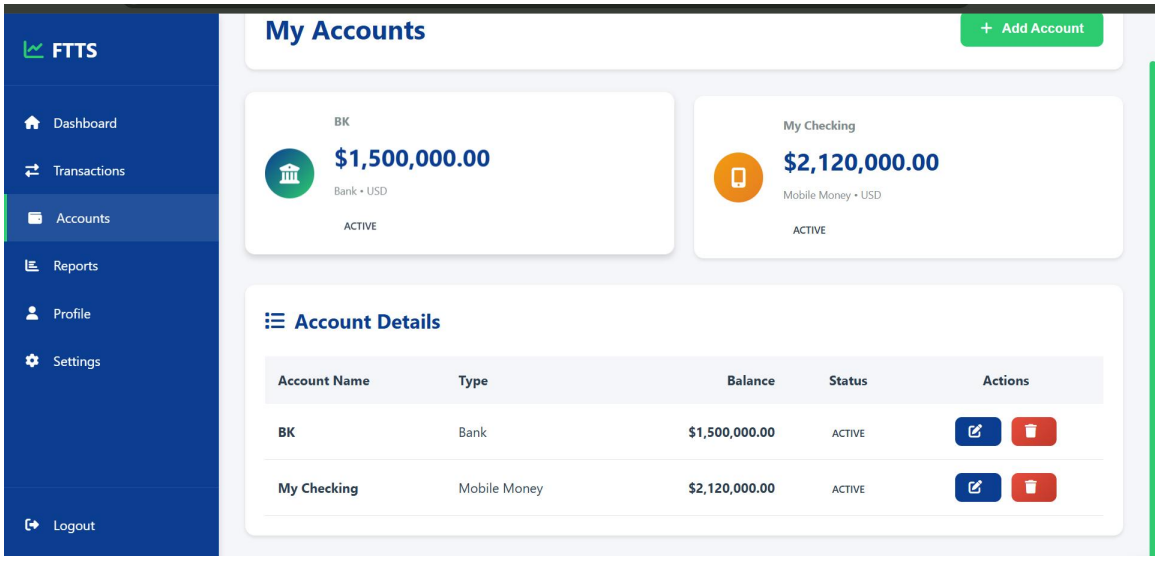


Select All Account I have

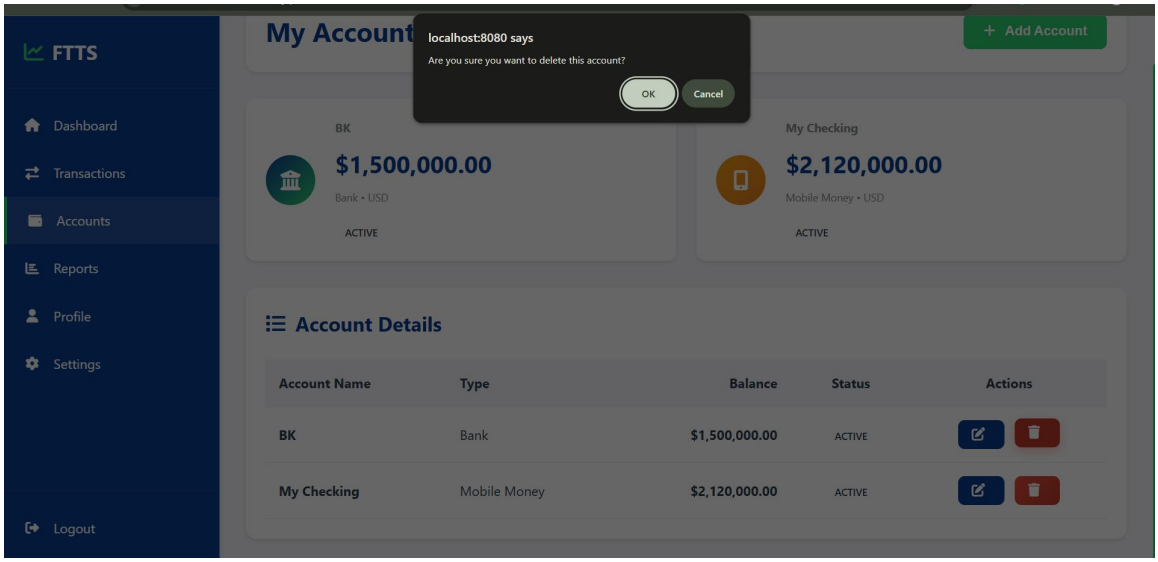


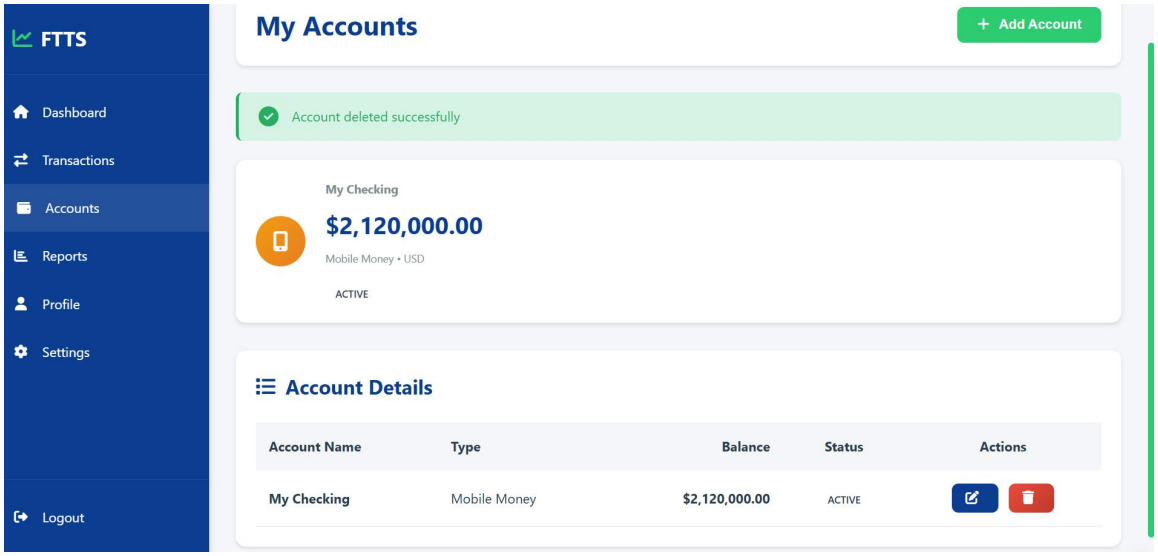
Delete BK Account

Before Delete Operation



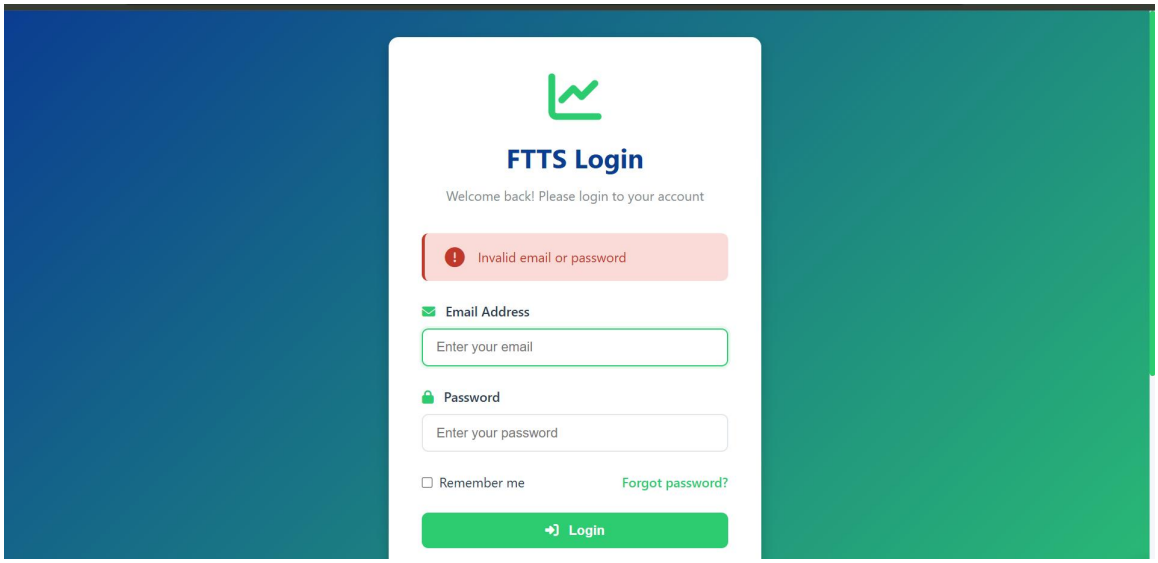
After Delete Operation





- Validation errors

Try To login With the wrong Data



SAMPLE CODE

DB CONNECTION

```
public class DatabaseUtil {  42 usages  DIVINEakisa
    private static String DB_DRIVER;  2 usages
    private static String DB_URL;  2 usages
    private static String DB_USERNAME;  2 usages
    private static String DB_PASSWORD;  2 usages

    static {
        loadDatabaseProperties();
    }

    /**
     * Load database properties from db.properties file
     */
    private static void loadDatabaseProperties() {  1 usage  DIVINEakisa
        Properties props = new Properties();
        try (InputStream input = DatabaseUtil.class.getClassLoader()
                .getResourceAsStream("db.properties")) {
            if (input != null) {
                props.load(input);
                DB_DRIVER = props.getProperty("db.driver");
                DB_URL = props.getProperty("db.url");
                DB_USERNAME = props.getProperty("db.username");
                DB_PASSWORD = props.getProperty("db.password");

                // Load JDBC driver
                Class.forName(DB_DRIVER);
            } else {
```

```
    private static void loadDatabaseProperties() {  1 usage   DIVINEakisa
                DB_DRIVER = props.getProperty("db.driver");
                DB_URL = props.getProperty("db.url");
                DB_USERNAME = props.getProperty("db.username");
                DB_PASSWORD = props.getProperty("db.password");

                // Load JDBC driver
                Class.forName(DB_DRIVER);
            } else {
                throw new RuntimeException("Unable to find db.properties");
            }
        } catch (IOException | ClassNotFoundException e) {
            throw new RuntimeException("Error loading database configuration", e);
        }
    }

    /**
     * Get database connection
     * @return Connection object
     * @throws SQLException if connection fails
     */
    public static Connection getConnection() throws SQLException {  37 usages   DIVINEakisa
        return DriverManager.getConnection(DB_URL, DB_USERNAME, DB_PASSWORD);
    }

    /**
     * Close database connection
     * @param connection Connection object to close
     */
    public static void closeConnection(Connection connection) {  no usages   DIVINEakisa
```

```
public class DatabaseUtil {  42 usages   DIVINEakisa
     * @param connection Connection object to close
     */
    public static void closeConnection(Connection connection) {  no usages   DIVINEakisa
        if (connection != null) {
            try {
                connection.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }

    /**
     * Test database connection
     * @return true if connection is successful
     */
    public static boolean testConnection() {  no usages   DIVINEakisa
        try (Connection conn = getConnection()) {
            return conn != null && !conn.isClosed();
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }
}
```

Create Account:

```java
public boolean createAccount(Account account) {  1 usage  DIVINEakisa
    String sql = "INSERT INTO accounts (user_id, account_name, account_type, balance, currency, status) " +
                 "VALUES (?, ?, ?, ?, ?, ?)";

    try (Connection conn = DatabaseUtil.getConnection();
         PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {

        stmt.setInt(1, account.getUserId());
        stmt.setString(2, account.getAccountName());
        stmt.setString(3, account.getAccountType());
        stmt.setBigDecimal(4, account.getBalance());
        stmt.setString(5, account.getCurrency());
        stmt.setString(6, account.getStatus());

        int rowsAffected = stmt.executeUpdate();

        if (rowsAffected > 0) {
            ResultSet rs = stmt.getGeneratedKeys();
            if (rs.next()) {
                account.setAccountId(rs.getInt(1));
            }
            return true;
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }
```

```java
        }

        return false;
    }

    /**
     * Get account by ID
     * @param accountId Account ID
     * @return Account object
     */
    public Account getAccountById(int accountId) {  1 usage  DIVINEakisa
        String sql = "SELECT * FROM accounts WHERE account_id = ?";

        try (Connection conn = DatabaseUtil.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, accountId);
            ResultSet rs = stmt.executeQuery();

            if (rs.next()) {
                return extractAccountFromResultSet(rs);
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }

        return null;
```

```java
public List<Account> getAccountsByUserId(int userId) {  no usages  DIVINEakisa
    List<Account> accounts = new ArrayList<>();
    String sql = "SELECT * FROM accounts WHERE user_id = ? ORDER BY created_at DESC";

    try (Connection conn = DatabaseUtil.getConnection();
         PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, userId);
        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            accounts.add(extractAccountFromResultSet(rs));
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return accounts;
}
```

Update:

```java
public boolean updateAccountStatus(int accountId, String status) {  no usages  DIVINEakisa
    String sql = "UPDATE accounts SET status = ?, updated_at = CURRENT_TIMESTAMP " +
                 "WHERE account_id = ?";

    try (Connection conn = DatabaseUtil.getConnection();
         PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, status);
        stmt.setInt(2, accountId);

        return stmt.executeUpdate() > 0;

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return false;
}
```

Delete:

```java
public boolean deleteAccount(int accountId) {  1 usage  DIVINEakisa
    String sql = "DELETE FROM accounts WHERE account_id = ?";

    try (Connection conn = DatabaseUtil.getConnection();
         PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, accountId);
        return stmt.executeUpdate() > 0;

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return false;
}
```

Select:

```java
public BigDecimal getTotalBalance(int userId) {  no usages  DIVINEakisa
    String sql = "SELECT SUM(balance) as total FROM accounts " +
                 "WHERE user_id = ? AND status = 'Active'";

    try (Connection conn = DatabaseUtil.getConnection();
         PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, userId);
        ResultSet rs = stmt.executeQuery();

        if (rs.next()) {
            BigDecimal total = rs.getBigDecimal("total");
            return total != null ? total : BigDecimal.ZERO;
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return BigDecimal.ZERO;
}
```

```java
public boolean verifyAccountOwnership(int accountId, int userId) {  3 usages  DIVINEakisa
    String sql = "SELECT COUNT(*) FROM accounts WHERE account_id = ? AND user_id = ?";

    try (Connection conn = DatabaseUtil.getConnection();
         PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, accountId);
        stmt.setInt(2, userId);
        ResultSet rs = stmt.executeQuery();

        if (rs.next()) {
            return rs.getInt(1) > 0;
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return false;
}
```

```
    private Account extractAccountFromResultSet(ResultSet rs) throws SQLException {  3 usages  DIVINEakisa
        Account account = new Account();
        account.setAccountId(rs.getInt("account_id"));
        account.setUserId(rs.getInt("user_id"));
        account.setAccountName(rs.getString("account_name"));
        account.setAccountType(rs.getString("account_type"));
        account.setBalance(rs.getBigDecimal("balance"));
        account.setCurrency(rs.getString("currency"));
        account.setStatus(rs.getString("status"));
        account.setCreatedAt(rs.getTimestamp("created_at"));
        account.setUpdatedAt(rs.getTimestamp("updated_at"));
        return account;
    }
}
```

7. Deployment Requirements

- Apache Tomcat server
- Project runs without errors
- Database connection configured
- WAR file for deployment